

Teacher-Anchored and Statistical Calibration for Drift-Adaptive Affine GKP Decoding

Abstract

Approximate Gottesman–Kitaev–Preskill (GKP) quantum error correction encodes a qubit into an oscillator and uses continuous-valued modular syndromes to infer displacement errors. In realistic finite-energy GKP states, syndrome statistics are shaped by state-preparation width, measurement noise, circuit-level imperfections, and slow calibration drift. This work studies a two-timescale decoding architecture for such settings. A low-latency fast loop applies an affine correction $\Delta_t = K_t s_t + b_t$, while a slower calibration loop estimates the current effective noise state from recent syndrome histograms and updates the runtime parameters. This keeps the per-shot path compatible with fixed-point hardware execution while allowing the decoder to adapt to nonstationary noise.

The architecture is designed around a narrow and deployment-oriented role for learning. A classical teacher estimator provides an interpretable baseline for the affine parameters, and a lightweight CNN predicts only a bounded residual calibration term, currently focused on the bias b_t . This differs from end-to-end neural decoders and from outer-code soft-information decoders: the learned module updates a low-dimensional physical-layer calibration surface, whereas the real-time path remains a deterministic affine rule. Software-HIL experiments over four drift scenarios show that adaptive affine calibration improves over filtering baselines, that histogram-delta features are important for the learned residual branch, and that a statistical calibration variant substantially reduces the logical-error proxy across the same drift suite. A local sensitivity study further indicates that the statistical-calibration advantage is not tied to a single narrow parameter choice.

Contents

1	Introduction	2
2	Summary of Contributions	3
3	Brief Review of the GKP Code	4
3.1	Ideal and approximate code states	4
3.2	Syndrome measurement as modular displacement information	4
3.3	Local affine decoding and its limits	4
3.4	Logical failure criterion	5
4	Noise and Drift Model	5
4.1	Effective noise state	5
4.2	Noise sources represented by the model	5
4.3	Drift scenarios	6

5	Model Architecture	6
5.1	Fast loop: affine fixed-point decoder	6
5.2	Parameter mapping	6
5.3	Teacher estimators	7
5.4	CNN residual branch	7
5.5	Statistical calibration branch	7
5.6	Stage-and-commit runtime contract	7
6	Relationship to Existing Work	8
6.1	GKP and bosonic analog soft information	8
6.2	Adaptive priors and syndrome-statistics estimation	8
6.3	Learned low-latency QEC modules	8
6.4	Real-time and FPGA QEC decoders	8
7	Experimental Setup	8
7.1	Software-HIL protocol	8
7.2	Scenarios and modes	8
8	Numerical Results and Benchmark Plan	9
8.1	Four-scenario affine benchmark	9
8.2	Feature and teacher ablations	9
8.3	Statistical calibration as a strong comparator	9
8.4	Local sensitivity of statistical calibration	10
8.5	Per-scenario best statistical calibration variant	10
8.6	Mechanism probe for residual-b behavior	11
8.7	Unseen drift generalization	11
8.8	Oracle and wrapped-Gaussian baselines	11
8.9	Runtime, quantization, and fixed-point degradation	11
8.10	Embedded runtime and board-level validation	11
9	Discussion	11
10	Conclusion	11

1 Introduction

Bosonic codes protect quantum information by encoding a logical qubit into the Hilbert space of an oscillator. The GKP code is a central example: ideal code states form a periodic lattice in phase space, so small shift errors can be diagnosed through modular syndrome measurements and corrected by displacement operations [1, 2]. Unlike binary stabilizer syndromes, GKP syndromes are analog. Their continuous values carry information not only about the likely displacement, but also about how close the correction is to a lattice decision boundary.

The practical challenge is that physical GKP states are approximate finite-energy states. Their comb peaks have finite width and an envelope, and syndrome measurements are further affected by measurement inefficiency, noisy auxiliary states, circuit-level imperfections, oscillator loss, and calibration error [2, 9]. Consequently, the effective noise model seen by the decoder can change over time. A fixed syndrome-to-correction map may be well tuned for one operating point but mismatched after bias, variance, or covariance drift.

Recent GKP and bosonic-decoding work makes clear that analog soft information is valuable. Surface-GKP and concatenated bosonic-QLDPC decoders use continuous syndrome information to set matching weights, belief-propagation messages, or effective outer-code priors [4, 5, 6, 7, 8]. At the same time, modern QEC decoder systems show that learned modules are most compelling when their latency role and integration boundary are explicit [15, 16]. Hardware and calibration-oriented studies similarly emphasize that decoder priors must be kept aligned with device statistics [10, 11, 12, 13, 14].

This work applies that systems principle at the physical GKP correction layer. The fast loop is intentionally simple:

$$\Delta_t = K_t s_t + b_t, \quad (1)$$

where s_t is the measured two-quadrature GKP syndrome and (K_t, b_t) are quantized runtime parameters. The slow loop observes recent syndrome histograms, estimates an effective noise state, and updates (K_t, b_t) at a lower rate. The design therefore separates two concerns: the per-shot correction path is a bounded affine operation, while the nonstationary estimation problem is handled asynchronously through calibrated parameter updates.

The main advantage of this formulation is its combination of physical structure, adaptation, and deployment discipline. Compared with a fixed affine decoder, the parameters can track drift. Compared with a full neural decoder, the online computation is smaller and more predictable. Compared with outer-code soft-information methods, the adaptation target is a physical-layer GKP displacement rule rather than a matching graph or LDPC message schedule. Compared with calibration-conditioned neural decoders, the slow context does not enlarge the per-shot path; it only changes the affine parameters staged into the runtime bank.

2 Summary of Contributions

The work can be organized around four contributions.

1. A two-timescale adaptive affine formulation for approximate GKP decoding. The per-shot correction path is formulated as a bounded affine estimator $\Delta_t = K_t s_t + b_t$. The expensive adaptation problem is moved into a slow loop that estimates the current effective noise state from recent syndrome statistics. This links the continuous-syndrome GKP picture to a runtime contract that can be implemented as a quantized matrix-vector operation.

2. A teacher-anchored residual calibration strategy. A classical teacher estimator produces a stable baseline $(K_t^{\text{teacher}}, b_t^{\text{teacher}})$. The CNN predicts a small residual,

$$\delta b_t = f_\phi(H_{t-c+1:t}, \Delta H_t, z_t^{\text{teacher}}), \quad (2)$$

and the committed parameters are

$$K_t = K_t^{\text{teacher}}, \quad b_t = \text{EMA}\left(b_t^{\text{teacher}} + \delta b_t\right). \quad (3)$$

Learning is therefore used to calibrate a low-dimensional control surface, not to replace the GKP decoder end-to-end.

3. A deployment-aware runtime architecture. The architecture exposes fixed-point quantization, clipping, saturation diagnostics, parameter smoothing, stale-parameter behavior, and double-buffered stage-and-commit updates. These constraints become measurable objects in the benchmark rather than assumptions added after the algorithm is designed.

4. A benchmark suite for adaptive affine calibration. The numerical study compares filtering baselines, learned residual variants, and statistical calibration variants under four drift scenarios. The results separate three questions: whether adaptive affine decoding improves over filtering baselines, which learned residual features matter, and whether a simple statistical calibration rule can provide a strong non-neural comparator inside the same runtime contract.

3 Brief Review of the GKP Code

3.1 Ideal and approximate code states

The square-lattice GKP code encodes a qubit into an oscillator by using periodic structure in the q and p quadratures [1]. In the convention used here, the lattice constant is

$$\lambda = \sqrt{2\pi}. \quad (4)$$

An ideal logical state may be represented heuristically as an infinite comb, for example

$$|\bar{0}\rangle \propto \sum_{n \in \mathbb{Z}} |n\lambda\rangle_q. \quad (5)$$

Ideal comb states are not physical because they require infinite energy. Approximate GKP states replace the infinitely sharp comb with finitely squeezed peaks and an envelope. Finite-energy structure is central to practical GKP QEC rather than a minor implementation detail [2]: it determines the intrinsic syndrome uncertainty and contributes an effective displacement-like noise.

3.2 Syndrome measurement as modular displacement information

Let the accumulated phase-space displacement error before correction be

$$e_t = \begin{bmatrix} e_{q,t} \\ e_{p,t} \end{bmatrix}. \quad (6)$$

An ideal GKP syndrome observes this displacement modulo the lattice:

$$s_t = e_t \bmod \lambda, \quad s_{q,t}, s_{p,t} \in [-\lambda/2, \lambda/2). \quad (7)$$

In a finite-energy and noisy-measurement setting, the measured syndrome is more accurately described as

$$\tilde{s}_t = \text{mod}(e_t, \lambda) + \eta_t^{\text{meas}} + \eta_t^{\text{GKP}}. \quad (8)$$

Here η_t^{meas} represents measurement and auxiliary-state contributions, while η_t^{GKP} summarizes finite-squeezing uncertainty. The decoder therefore observes a noisy modular representative, not the absolute displacement.

3.3 Local affine decoding and its limits

The modulo structure makes exact GKP decoding nonlinear and branch dependent. Near a single lattice branch, however, a Gaussian approximation gives a useful local model. If e and s are jointly Gaussian,

$$\begin{bmatrix} e \\ s \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \mu_e \\ \mu_s \end{bmatrix}, \begin{bmatrix} \Sigma_{ee} & \Sigma_{es} \\ \Sigma_{se} & \Sigma_{ss} \end{bmatrix} \right), \quad (9)$$

then the linear-MMSE estimate is

$$\hat{e} = \mu_e + \Sigma_{es}\Sigma_{ss}^{-1}(s - \mu_s) = Ks + b, \quad (10)$$

where

$$K = \Sigma_{es}\Sigma_{ss}^{-1}, \quad b = \mu_e - K\mu_s. \quad (11)$$

This derivation explains why an affine fast path is a plausible low-latency approximation. It also marks the boundary of the approximation. Near lattice decision boundaries, the posterior can be multimodal; maximum-likelihood, closest-lattice-point, or wrapped-Gaussian decoders can outperform a single global affine rule [3, 9].

3.4 Logical failure criterion

After correction, the residual displacement is wrapped into the GKP fundamental cell. A logical error occurs when the residual crosses a logical decision boundary, for example

$$|r_{q,t}| > \lambda/2 \quad \Rightarrow \quad X_L \text{ error}, \quad (12)$$

and similarly for the p -quadrature. Closed-loop logical error probability is therefore the central metric; parameter-regression error is useful only when it correlates with the downstream correction outcome.

4 Noise and Drift Model

4.1 Effective noise state

The slow loop does not attempt to infer every microscopic hardware parameter. It uses a low-dimensional effective state,

$$\theta_t^{\text{noise}} = (\sigma_t, \mu_{q,t}, \mu_{p,t}, \vartheta_t), \quad (13)$$

where σ_t controls the displacement scale, $\mu_{q,t}$ and $\mu_{p,t}$ represent mean bias, and ϑ_t captures covariance-axis rotation. This compact parameterization can be mapped to affine runtime parameters and estimated from finite histogram windows.

4.2 Noise sources represented by the model

The model is an effective displacement-noise model with additional measurement uncertainty. It absorbs several physical effects:

1. finite-energy GKP peak width and envelope effects;
2. Gaussian random displacement noise in q and p ;
3. biased displacement means caused by calibration offsets;
4. anisotropic or rotated covariance caused by asymmetric noise;
5. syndrome measurement noise and noisy auxiliary-state contributions.

This is narrower than a full circuit-level GKP noise model, but it is aligned with the affine fast-path contract.

4.3 Drift scenarios

The benchmark suite uses four drift scenarios:

1. `static_bias_theta`: a biased and rotated operating point;
2. `linear_ramp`: slowly changing effective noise parameters;
3. `step_sigma_theta`: a sudden variance/orientation change;
4. `periodic_drift`: periodic nonstationarity.

These scenarios separate different adaptation demands: steady calibration, tracking, shock response, and periodic following.

5 Model Architecture

5.1 Fast loop: affine fixed-point decoder

The fast loop receives the current measured syndrome s_t , reads the active parameter bank, and computes

$$s_t^{\text{clip}} = \text{clip}(s_t, -s_{\max}, s_{\max}), \quad (14)$$

$$\Delta_t^{\text{raw}} = K_t s_t^{\text{clip}} + b_t, \quad (15)$$

$$\Delta_t = Q(\text{clip}(\Delta_t^{\text{raw}}, -\Delta_{\max}, \Delta_{\max})), \quad (16)$$

where $Q(\cdot)$ denotes fixed-point quantization. The implementation uses an FPGA-oriented Q4.20 representation for syndrome values and runtime parameters. The fast loop also records diagnostic counters, including histogram-input saturation, correction saturation, overflow, aggressive parameter events, commit counts, and fallback-related signals.

5.2 Parameter mapping

Given an estimated noise state, the parameter mapper constructs an error covariance

$$C = R(\vartheta) \begin{bmatrix} \sigma_q^2 & 0 \\ 0 & \sigma_p^2 \end{bmatrix} R(\vartheta)^\top, \quad (17)$$

and a measurement covariance

$$R_{\text{meas}} = (\sigma_{\text{meas}}^2 + \Delta_{\text{eff}}^2)I. \quad (18)$$

The raw gain is

$$K_{\text{raw}} = C(C + R_{\text{meas}})^{-1}, \quad (19)$$

followed by gain clipping or scaling. The bias target is

$$b_{\text{target}} = \alpha(I - K_{\text{target}})\mu, \quad \mu = \begin{bmatrix} \mu_q \\ \mu_p \end{bmatrix}. \quad (20)$$

Finally, exponential smoothing produces staged parameters:

$$K_t = (1 - \beta)K_{t-1} + \beta K_{\text{target}}, \quad b_t = (1 - \beta)b_{t-1} + \beta b_{\text{target}}. \quad (21)$$

5.3 Teacher estimators

The teacher family provides stable estimates of θ_t^{noise} from recent syndrome history. A simple teacher computes moments over a histogram window. Stronger teachers such as EKF, UKF, RLS, or particle-filter variants add state-space assumptions over time. Abstractly,

$$\hat{\theta}_t^{\text{teacher}} = \text{Teacher}(H_{1:t}), \quad (22)$$

and

$$(K_t^{\text{teacher}}, b_t^{\text{teacher}}) = \text{ParamMapper}(\hat{\theta}_t^{\text{teacher}}). \quad (23)$$

The teacher keeps the learned branch near an interpretable calibration surface and creates meaningful ablation baselines.

5.4 CNN residual branch

The CNN consumes a short context of normalized syndrome histograms and histogram deltas:

$$X_t^{\text{hist}} = [H_{t-c+1}, \dots, H_t, \Delta H_{t-c+2}, \dots, \Delta H_t]. \quad (24)$$

Teacher-side features may include

$$z_t^{\text{teacher}} = (\hat{\theta}_t^{\text{teacher}}, K_t^{\text{teacher}}, b_t^{\text{teacher}}, \Delta b_t^{\text{teacher}}, \dots). \quad (25)$$

The gated branch narrows this to a small set of teacher- b and teacher- Δb scalars. The learned branch predicts

$$\hat{\delta b}_t = f_\phi(X_t^{\text{hist}}, z_t^{\text{teacher}}), \quad (26)$$

then clips the result:

$$\delta b_t = \text{clip}(s_b \hat{\delta b}_t, -b_{\max}, b_{\max}). \quad (27)$$

The committed parameter is

$$K_t = K_t^{\text{teacher}}, \quad b_t = \text{EMA}(b_t^{\text{teacher}} + \delta b_t). \quad (28)$$

5.5 Statistical calibration branch

The same affine runtime contract also supports a non-neural statistical calibration branch. This branch estimates a bounded bias correction directly from recent syndrome statistics and emits runtime parameters through the same clipping, smoothing, and stage-and-commit path. Its role is both practical and scientific: it tests whether the benefit comes from a general histogram-driven calibration principle rather than from the CNN architecture alone.

5.6 Stage-and-commit runtime contract

The slow loop does not directly mutate active fast-loop parameters. It stages a candidate (K, b) into an inactive bank and commits it at a safe epoch boundary:

$$(K_t, b_t) = \begin{cases} (K^A, b^A), & t < t_{\text{commit}}, \\ (K^B, b^B), & t \geq t_{\text{commit}}. \end{cases} \quad (29)$$

This contract allows stale-parameter effects, update latency, commit success, rollback/fallback behavior, and fixed-point stability to be measured separately from logical performance.

6 Relationship to Existing Work

6.1 GKP and bosonic analog soft information

GKP analog syndrome information has already been shown to be valuable. Work on analog QEC, surface-GKP codes, and concatenated bosonic-QLDPC decoding uses continuous measurement information to set decoder weights or soft messages [3, 4, 5, 6, 7, 8]. The distinction here is that the analog statistics are compressed into physical-layer affine GKP fast-path parameters, rather than into outer-code matching weights or LDPC messages.

6.2 Adaptive priors and syndrome-statistics estimation

Adaptive-weight and syndrome-statistics methods estimate decoder weights or noise parameters from measured data [10, 11, 12]. Calibrated decoders and decoder-prior optimization show that prior mismatch can materially affect logical performance [13, 14]. This work borrows that lesson, but maps the adaptation target to the GKP affine parameters (K, b) and constrains the update through a stage-and-commit runtime contract.

6.3 Learned low-latency QEC modules

AI pre-decoders, high-accuracy neural decoders, and calibration-conditioned FiLM decoders demonstrate a systems-oriented pattern: a learned module is most useful when its input/output contract, latency role, and integration with a classical decoder are explicit [15, 16, 17]. The learned module here is not the per-shot decoder. It is a slow calibration component that updates low-dimensional affine parameters; the per-shot path remains fixed-point affine logic.

6.4 Real-time and FPGA QEC decoders

Real-time QEC decoders and FPGA-integrated systems have reported low-latency hardware decoding in stabilizer-code settings [18, 19, 20, 21]. Real-time bosonic QEC has also been demonstrated in experimental feedback systems [22, 23]. These works set the standard for full deployment claims: closed-loop timing, worst-case latency, resource use, and hardware integration. The contribution here is a hardware-compatible GKP affine calibration contract and a software-HIL evaluation of its drift-adaptive behavior.

7 Experimental Setup

7.1 Software-HIL protocol

The numerical experiments use a software-HIL protocol that preserves the fast loop, slow loop, parameter-bank, and staged-commit interfaces while executing the underlying experiment in software. This protocol is sufficient for comparing adaptive calibration laws under identical drift trajectories and paired seeds. It does not measure board-level latency or hardware resource use.

7.2 Scenarios and modes

The four drift scenarios are:

Scenario key	Interpretation
<code>static.bias.theta</code>	biased and rotated operating point
<code>linear.ramp</code>	slowly changing effective noise parameters
<code>step.sigma.theta</code>	sudden variance/orientation change
<code>periodic.drift</code>	periodic nonstationarity

The main comparison includes filtering baselines, residual affine baselines, the learned hybrid residual branch, and statistical calibration variants. The reported metric is `final_ler_mean`, where lower values indicate fewer logical failures under the fixed software-HIL protocol.

8 Numerical Results and Benchmark Plan

8.1 Four-scenario affine benchmark

Table 1 compares five affine decoding modes under the four drift scenarios. The learned hybrid residual branch is the best mode among these five in all four scenarios, with UKF as the strongest filtering baseline.

Scenario	EKF	UKF	Const.- μ	RLS- b	Hybrid- b
<code>static.bias.theta</code>	0.838110	0.825370	0.836658	0.837577	0.810902
<code>linear.ramp</code>	0.819200	0.811201	0.816911	0.819373	0.787755
<code>step.sigma.theta</code>	0.822365	0.811548	0.819784	0.821493	0.788800
<code>periodic.drift</code>	0.832192	0.821558	0.829670	0.832334	0.806392

Table 1: Four-scenario affine benchmark. Entries are `final_ler_mean`; lower is better.

8.2 Feature and teacher ablations

Table 2 isolates the learned residual branch. Removing histogram deltas degrades performance relative to the full hybrid model, indicating that temporal changes in the syndrome histogram are useful for tracking drift. Removing teacher prediction also degrades the full hybrid model, but the no-teacher-params variant performs best in this ablation set. This result argues against a simple story in which every teacher-derived channel is uniformly beneficial.

Mode	Avg. LER	Δ vs UKF	Δ vs Hybrid Full
<code>ukf</code>	0.817382	0.000000	+0.018837
<code>hybrid.full</code>	0.798545	-0.018837	0.000000
<code>hybrid.no.hist.deltas</code>	0.826723	+0.009341	+0.028178
<code>hybrid.no.teacher.prediction</code>	0.807251	-0.010131	+0.008706
<code>hybrid.no.teacher.params</code>	0.749621	-0.067761	-0.048924
<code>hybrid.no.teacher.deltas</code>	0.800329	-0.017053	+0.001784

Table 2: Feature and teacher ablation summary. Negative Δ means improvement over the reference.

8.3 Statistical calibration as a strong comparator

Table 3 adds a statistical calibration lane to the same four-scenario suite. The statistical calibration branch beats both the hybrid residual branch and UKF in every scenario. This is an important

result for positioning the work: the central object is not only a CNN residual model, but a broader adaptive affine calibration contract in which non-neural statistical estimators can also be strong.

Scenario	UKF	Hybrid- <i>b</i>	StatCalib
<code>static_bias_theta</code>	0.825370	0.810902	0.431708
<code>linear_ramp</code>	0.811201	0.787755	0.467083
<code>step_sigma_theta</code>	0.811548	0.788800	0.460016
<code>periodic_drift</code>	0.821558	0.806392	0.438751

Table 3: Statistical calibration comparison under the same drift suite. Entries are `final_ler_mean`; lower is better.

8.4 Local sensitivity of statistical calibration

The statistical calibration branch was evaluated on a five-point local grid around the default parameter setting: default scale/clip/threshold, low residual scale, high residual scale, low residual clip, and high signal threshold. Table 4 shows that the advantage is not tied to a single parameter point. The high-threshold variant has the best aggregate mean LER by a very small margin, whereas the default variant wins three of four scenarios and has the best mean within-statcalib rank.

Variant	Mean LER	Mean Rank	Scenario Wins
<code>statcalib_high_threshold</code>	0.449241	1.75	1
<code>statcalib_default</code>	0.449254	1.25	3
<code>statcalib_high_scale</code>	0.456477	3.00	0
<code>statcalib_low_clip</code>	0.481484	4.50	0
<code>statcalib_low_scale</code>	0.485129	4.50	0

Table 4: Local sensitivity of statistical calibration. Mean rank is computed within the five statistical calibration variants for each scenario.

8.5 Per-scenario best statistical calibration variant

Table 5 reports the best statistical calibration variant in each scenario. Every scenario-best variant clears both the UKF and hybrid residual baselines by a large margin. The `static_bias_theta` winner is the only row with a mixed status flag, so that scenario is best treated as a useful but locally caveated win.

Scenario	Best variant	Best LER	Gap vs Hybrid- <i>b</i>
<code>static_bias_theta</code>	<code>high_threshold</code>	0.431684	0.379889
<code>linear_ramp</code>	<code>default</code>	0.467094	0.321178
<code>step_sigma_theta</code>	<code>default</code>	0.458834	0.328883
<code>periodic_drift</code>	<code>default</code>	0.439352	0.367232

Table 5: Best statistical calibration variant per scenario. Gap is the hybrid residual LER minus the best statistical calibration LER.

8.6 Mechanism probe for residual-b behavior

A multi-seed mechanism figure is available for the learned residual branch. It compares seed-wise hybrid-model gaps and a lower-clip intervention. The descriptive reading is that residual-b high-amplitude behavior appears in more than one seed, but lowering the residual clip is mixed and often harmful. This supports a cautious interpretation: residual amplitude participates in some failure modes, but it is not a monotonic causal explanation by itself.

8.7 Unseen drift generalization

The current four scenarios cover steady bias, ramp tracking, step response, and periodic drift. A stronger benchmark requires unseen drift families such as random-walk drift, $1/f$ -like drift, burst measurement noise, coupled bias-variance drift, and faster-than-window drift. This experiment is needed to distinguish genuine drift adaptation from overfitting to a small set of hand-designed trajectories.

8.8 Oracle and wrapped-Gaussian baselines

The affine methods need comparison against stronger theoretical baselines: nearest-lattice hard decoding, static affine decoding, oracle affine decoding with true noise state, and wrapped-Gaussian or maximum-likelihood decoding in small settings. These baselines quantify how much performance is lost because the fast path is affine, and how much is lost because the slow loop estimates the noise state imperfectly.

8.9 Runtime, quantization, and fixed-point degradation

The deployment claim depends on more than logical performance. The benchmark requires fixed-point versus floating-point comparisons, saturation and overflow rates, commit latency, stale-parameter penalty, fallback frequency, and per-update slow-loop cost. These measurements connect the mathematical affine rule to the real-time decoder contract.

8.10 Embedded runtime and board-level validation

Embedded inference and board-level validation are separate from software-HIL simulation. The relevant measurements are exported-model equivalence, embedded runtime latency, host-to-device update cost, hardware resource use, and closed-loop timing. These experiments are needed before replacing “hardware-compatible” with a stronger hardware-demonstrated claim.

9 Discussion

10 Conclusion

This work frames drift-adaptive GKP correction as a two-timescale affine calibration problem. The fast loop remains a bounded fixed-point rule, $\Delta_t = K_t s_t + b_t$, while a slower loop updates the affine parameters from recent syndrome statistics. This formulation creates a practical middle ground between fixed decoders, which cannot track hardware drift, and full neural decoders, which can be difficult to place on a deterministic per-shot path.

The numerical results support two main conclusions. First, adaptive affine decoding with a teacher-anchored residual branch improves over filtering baselines in the four-scenario drift suite,

and ablations show that temporal histogram changes carry useful information. Second, statistical calibration is a strong comparator under the same runtime contract; its advantage persists under a local sensitivity grid. Together, these results suggest that the most robust scientific story is not simply that a CNN is better than classical filters, but that low-dimensional histogram-driven calibration of an affine GKP fast path is a promising structure for drift-adaptive decoding.

The remaining technical gap is to connect this promising software-HIL evidence to broader drift families, stronger oracle and wrapped-Gaussian baselines, and measured runtime constraints. Those additions would test whether the affine calibration contract remains useful when the benchmark moves from a controlled drift suite toward a deployment-grade decoder evaluation.

References

- [1] D. Gottesman, A. Kitaev, and J. Preskill, “Encoding a qubit in an oscillator,” *Physical Review A*, 2001.
- [2] A. L. Grimsmo and S. Puri, “Quantum Error Correction with the Gottesman–Kitaev–Preskill Code,” *PRX Quantum* 2, 020101, 2021.
- [3] K. Fukui, A. Tomita, A. Okamoto, and K. Fujii, “High-threshold fault-tolerant quantum computation with analog quantum error correction,” *Physical Review X*, 2018.
- [4] K. Noh and C. Chamberland, “Fault-tolerant bosonic quantum error correction with the surface-GKP code,” *Physical Review A*, 2020.
- [5] K. Noh, C. Chamberland, and F. G. S. L. Brandao, “Low-overhead fault-tolerant quantum error correction with the surface-GKP code,” *PRX Quantum*, 2022.
- [6] N. Raveendran, N. Rengaswamy, F. Rozpedek, A. Raina, L. Jiang, and B. Vasic, “Finite-rate QLDPC-GKP coding scheme that surpasses the CSS Hamming bound,” *Quantum*, 2022.
- [7] L. Berent et al., “Analog information decoding of bosonic quantum LDPC codes,” *PRX Quantum*, 2024.
- [8] S. K. Borah, A. K. Pradhan, N. Raveendran, M. Pacenti, and B. Vasic, “Fault tolerant decoding of QLDPC-GKP codes with circuit level soft information,” preprint, 2025.
- [9] M.-A. Roy, T. Pousset, and B. Royer, “Decoding multimode Gottesman–Kitaev–Preskill codes with noisy auxiliary states,” preprint, 2025.
- [10] S. Spitz, B. Tarasinski, C. W. J. Beenakker, and T. E. O’Brien, “Adaptive weight estimator for quantum error correction,” *Advanced Quantum Technologies*, 2018.
- [11] T. Wagner, H. Kampermann, and D. Bruß, “Optimal noise estimation from syndrome statistics of quantum codes,” *Physical Review Research*, 2021.
- [12] H. Chen et al., “DGR: Tackling drifted and correlated noise in quantum error correction via decoding graph re-weighting,” preprint, 2023.
- [13] E. H. Chen et al., “Calibrated decoders for experimental quantum error correction,” *Physical Review Letters*, 2022.

- [14] V. Sivak, M. Newman, and P. Klimov, “Optimization of decoder priors for accurate quantum error correction,” *Physical Review Letters* 133, 150603, 2024.
- [15] C. Chamberland, J. Olle, M. Li, S. Thornton, and I. Baratta, “Fast and accurate AI-based pre-decoders for surface codes,” arXiv:2604.12841, 2026.
- [16] S. Stein, S. Kan, C. Liu, A. Harkness, S. Garner, Z. Du, Y. Ding, Y. Mao, and A. Li, “Calibration-conditioned FiLM decoders for low-latency decoding of quantum error correction evaluated on IBM repetition-code experiments,” preprint, 2026.
- [17] Google Quantum AI and collaborators, “Learning high-accuracy error decoding for quantum processors,” preprint, 2023.
- [18] L. Caune et al., “Demonstrating real-time and low-latency quantum error correction with superconducting qubits,” preprint, 2024.
- [19] X. Yang et al., “Real-time surface-code error correction using an FPGA-based neural-network decoder,” preprint, 2026.
- [20] M. Ziad et al., “Local clustering decoder: a fast and adaptive hardware decoder for the surface code,” preprint, 2024.
- [21] L. Maurer et al., “Real-time decoding of the gross code memory with FPGAs,” preprint, 2025.
- [22] V. Sivak et al., “Real-time quantum error correction beyond break-even,” *Nature*, 2023.
- [23] D. Lachance-Quirion et al., “Autonomous quantum error correction of Gottesman–Kitaev–Preskill states,” *Physical Review Letters*, 2024.